

In [3]: #1.

```
import numpy as np

# Create the first numpy array
array1 = np.array([1, 2, 3, 0])

# Create the second numpy array
array2 = np.array([4, 1, 0, 6])

# Addition
addition_result = array1 + array2
print("Addition: ", addition_result)

# Subtraction
subtraction_result = array1 - array2
print("Subtraction: ", subtraction_result)

# Multiplication
multiplication_result = array1 * array2
print("Multiplication: ", multiplication_result)

# Division
# To handle division by zero, we will use np.seterr to suppress warnings and handle divide-by-zero situations
np.seterr(divide='ignore', invalid='ignore')
division_result = array1 / array2
print("Division: ", division_result)
```

```
Addition: [5 3 3 6]
Subtraction: [-3  1  3 -6]
Multiplication: [4 2 0 0]
Division: [0.25 2.   inf 0.  ]
```

In [5]: #2.

```
import numpy as np

# Create the first numpy array (1-D) of random integers between 1 and 100 with 5 elements
```

```

array1 = np.random.randint(1, 101, size=5)
print("1D Array (Random Integers):", array1)

# Create the second numpy array (2-D) of random integers between 1 and 100 with shape 4x5
array2 = np.random.randint(1, 101, size=(4, 5))
print("\n2D Array (Random Integers with Shape 4x5):")
print(array2)

# Addition operation: Since array1 is 1D, it will be broadcasted to match the shape of array2
addition_result = array1 + array2
print("\nAddition (1D + 2D):")
print(addition_result)

# Subtraction operation: Subtracting 1D array from the 2D array
subtraction_result = array1 - array2
print("\nSubtraction (1D - 2D):")
print(subtraction_result)

```

1D Array (Random Integers): [21 88 58 61 96]

2D Array (Random Integers with Shape 4x5):

```

[[100  10  78  85  76]
 [ 31  57  52  72   1]
 [ 25  92  71  44  52]
 [ 34  51  18   3  46]]

```

Addition (1D + 2D):

```

[[121  98 136 146 172]
 [ 52 145 110 133  97]
 [ 46 180 129 105 148]
 [ 55 139  76  64 142]]

```

Subtraction (1D - 2D):

```

[[-79  78 -20 -24  20]
 [-10  31   6 -11  95]
 [ -4  -4 -13  17  44]
 [-13  37  40  58  50]]

```

In [9]: #3.

```

import numpy as np

```

```
# Create a 3D array of shape 3x3x3 with random integers between 1 and 100
array_3d = np.random.randint(1, 101, size=(3, 3, 3))

# Print the created 3D array
print("3D Array:")
print(array_3d)

# 1. Third column of the second 2D array (array_3d[1] corresponds to the second 2D array)
third_column_second_2d = array_3d[1, :, 2]
print("\nThird column of second 2-D array:")
print(third_column_second_2d)

# 2. First and third row of the first 2-D array (array_3d[0] corresponds to the first 2D array)
first_and_third_row_first_2d = array_3d[0, [0, 2], :]
print("\nFirst and third row of the first 2-D array:")
print(first_and_third_row_first_2d)

# 3. Intersection of the first row and third column in the third 2-D array
intersection_first_row_third_column_third_2d = array_3d[2, 0, 2]
print("\nIntersection of first row and third column in third 2-D array:")
print(intersection_first_row_third_column_third_2d)
```

3D Array:

```
[[[77 26 59]
   [54  7 19]
   [99 51 88]]
```

```
[[57 68 69]
 [83  8 42]
 [80 14  8]]
```

```
[[95 39 66]
 [ 4 52 88]
 [80  9 57]]]
```

Third column of second 2-D array:

```
[69 42  8]
```

First and third row of the first 2-D array:

```
[[77 26 59]
 [99 51 88]]
```

Intersection of first row and third column in third 2-D array:

```
66
```

In [11]: #4.

```
import numpy as np

# Create a 2D array with random integers between 1 and 100 and shape (5, 4)
array_2d = np.random.randint(1, 101, size=(5, 4))

# Print the created 2D array
print("2D Array:")
print(array_2d)

# 1. Print only the even numbers from the array
even_numbers = array_2d[array_2d % 2 == 0]
print("\nEven Numbers in the Array:")
print(even_numbers)

# 2. Print only the odd numbers from the array
```

```
odd_numbers = array_2d[array_2d % 2 != 0]
print("\nOdd Numbers in the Array:")
print(odd_numbers)

# 3. Print row-wise sum of all elements
row_wise_sum = np.sum(array_2d, axis=1)
print("\nRow-wise Sum of All Elements:")
print(row_wise_sum)

# 4. Print column-wise sum of all elements
column_wise_sum = np.sum(array_2d, axis=0)
print("\nColumn-wise Sum of All Elements:")
print(column_wise_sum)

# 5. Convert the 2D array into a 3D array of shape (2, 2, 5)
array_3d = array_2d.reshape(2, 2, 5)
print("\nConverted 3D Array (Shape 2x2x5):")
print(array_3d)
```

2D Array:

```
[[ 48  13  32  64]
 [ 61   2  67  72]
 [ 53  73  70  13]
 [ 41  59 100  66]
 [ 79  25  79  18]]
```

Even Numbers in the Array:

```
[ 48  32  64   2  72  70 100  66  18]
```

Odd Numbers in the Array:

```
[13 61 67 53 73 13 41 59 79 25 79]
```

Row-wise Sum of All Elements:

```
[157 202 209 266 201]
```

Column-wise Sum of All Elements:

```
[282 172 348 233]
```

Converted 3D Array (Shape 2x2x5):

```
[[[ 48  13  32  64  61]
   [  2  67  72  53  73]]

 [[ 70  13  41  59 100]
   [ 66  79  25  79  18]]]
```

In [13]:

```
#5.
import numpy as np

# Create a 2D array with random integers between 1 and 100 and shape (5, 4)
array_2d = np.random.randint(1, 101, size=(5, 4))

# Print the original 2D array
print("Original 2D Array:")
print(array_2d)

# 1. Swap first row with third row
array_2d[[0, 2]] = array_2d[[2, 0]]
print("\nAfter Swapping First and Third Row:")
```

```
print(array_2d)

# 2. Swap second column with fourth column
array_2d[:, [1, 3]] = array_2d[:, [3, 1]]
print("\nAfter Swapping Second and Fourth Column:")
print(array_2d)

# 3. Replace the values less than 50 with zero
array_2d[array_2d < 50] = 0
print("\nAfter Replacing Values Less Than 50 with Zero:")
print(array_2d)

# 4. Convert the 2D array into a 1D array
array_1d = array_2d.flatten()
print("\nConverted 1D Array:")
print(array_1d)
```

Original 2D Array:

```
[[95 12 71 96]
 [70  7 62 27]
 [75  1 78 93]
 [67 71 79 20]
 [28 42  2 42]]
```

After Swapping First and Third Row:

```
[[75  1 78 93]
 [70  7 62 27]
 [95 12 71 96]
 [67 71 79 20]
 [28 42  2 42]]
```

After Swapping Second and Fourth Column:

```
[[75 93 78  1]
 [70 27 62  7]
 [95 96 71 12]
 [67 20 79 71]
 [28 42  2 42]]
```

After Replacing Values Less Than 50 with Zero:

```
[[75 93 78  0]
 [70  0 62  0]
 [95 96 71  0]
 [67  0 79 71]
 [ 0  0  0  0]]
```

Converted 1D Array:

```
[75 93 78  0 70  0 62  0 95 96 71  0 67  0 79 71  0  0  0  0]
```

In []: #6.

Concept of Stacking in NumPy:

Stacking refers to the process of joining multiple arrays along a new axis. It is essentially a method of adding a

Examples:

#Vertical Stacking:

```
import numpy as np
```



```

# Two 2D arrays
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])

# Vertical stacking
result = np.vstack((arr1, arr2))

print("Vertical Stack:")
print(result)

#Horizontal Stacking:
# Horizontal stacking
result = np.hstack((arr1, arr2))

print("\nHorizontal Stack:")
print(result)

#Depth Stacking:
# Depth stacking
result = np.dstack((arr1, arr2))

print("\nDepth Stack:")
print(result)

```

Difference between Stacking and Concatenation

Axis of Operation:

Stacking: Stacks along a new axis (e.g., `np.vstack()` adds rows, `np.hstack()` adds columns, `np.dstack()` adds depth).

Concatenation: Combines along an existing axis (e.g., `np.concatenate()` merges arrays along specified axis).

New Dimension vs. Existing Dimension:

Stacking: Adds a new dimension to the arrays. For example, stacking creates a higher-dimensional array.

Concatenation: Merges arrays along an existing axis without adding a new dimension.

Shape Compatibility:

Stacking: Can stack arrays of different shapes as long as the dimensions are compatible for the stacking operation

Concatenation: Requires that the arrays have the same shape along all axes except for the one being concatenated.

